

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

Creating a Dropdown with Nested Menu Items using React MUI

Implementing Google Doc's Nested Dropdown Menu in ReactJS



Utkarsha Bakshi · [Follow](#)

Published in Geek Culture

2 min read · Apr 5, 2023



Listen



Share

... More

FILE VIEW INSERT TOOLS MORE

Image ▶

CSV ▶

As Table ▶

UPLOAD FROM COMPUTER

From URL

Dropdown with nested menu

This short article will teach us how to create a dropdown with nested menu items using React [Material UI](#). The goal is to create a dropdown with nested menu items similar to the Google Doc menu bar. Also, we will learn how to create a generic component that can contain any number of nested menus so that you can adapt it for your use case.

TLDR

Code sandbox with complete working code.

Prerequisites

- [Install and configure MUI in your React app](#)

Step 1: Define a nested menu item

First, let us define a generic nested menu item that we will use later for creating the dropdowns. Create a React component called `NestedMenuItem.tsx` with the following contents.

```
1  import * as React from 'react';
2  import Menu from '@mui/material/Menu';
3  import MenuItem from '@mui/material/MenuItem';
4  import ArrowRight from '@mui/icons-material/ArrowRight';
5
6  const NestedMenuItem = React.forwardRef((props, ref) => {
7    const {
8      parentMenuOpen,
9      label,
10     rightIcon = <ArrowRight style={{ fontSize: 16 }} />,
11     keepOpen,
12     children,
13     customTheme,
14     className,
15     tabIndex: tabIndexProp,
16     ContainerProps: ContainerPropsProp = {},
17     rightAnchored,
18     ...MenuItemProps
19   } = props;
20
21   const { ref: containerRefProp, ...ContainerProps } =
22     ContainerPropsProp;
23
24   const menuItemRef = React.useRef(null);
25   React.useImperativeHandle(ref, () => menuItemRef.current);
26
27   const containerRef = React.useRef(null);
28   React.useImperativeHandle(
29     containerRefProp,
30     () => containerRef.current
31   );
32
33   const menuContainerRef = React.useRef(null);
34
35   const [isSubMenuOpen, setIsSubMenuOpen] = React.useState(false);
36
37   const handleMouseEnter = (event) => {
38     setIsSubMenuOpen(true);
39
40     if (ContainerProps?.onMouseEnter) {
41       ContainerProps.onMouseEnter(event);
42     }
43   };
44
45   const handleMouseLeave = (event) => {
```

```
46         setIsSubMenuOpen(false);
47
48         if (ContainerProps?.onMouseLeave) {
49             ContainerProps.onMouseLeave(event);
50         }
51     };
52
53     const isSubMenuFocused = () => {
54         const active = containerRef.current?.ownerDocument?.activeElement;
55
56         for (const child of menuContainerRef.current?.children ?? []) {
57             if (child === active) {
58                 return true;
59             }
60         }
61         return false;
62     };
63
64     const handleFocus = (event) => {
65         if (event.target === containerRef.current) {
66             setIsSubMenuOpen(true);
67         }
68
69         if (ContainerProps?.onFocus) {
70             ContainerProps.onFocus(event);
71         }
72     };
73
74     const handleKeyDown = (event) => {
75         if (event.key === 'Escape') {
76             return;
77         }
78
79         if (isSubMenuFocused()) {
80             event.stopPropagation();
81         }
82
83         const active = containerRef.current?.ownerDocument?.activeElement;
84
85         if (event.key === 'ArrowLeft' && isSubMenuFocused()) {
86             containerRef.current?.focus();
87         }
88
89         if (
90             event.key === 'ArrowRight' &&
91             event.target === containerRef.current &&
92             event.target === active
```

```
92         event.target === active
93     ) {
94         const firstChild = menuContainerRef.current?.children[0];
95         firstChild?.focus();
96     }
97 };
98
99 const open = isSubMenuOpen && parentMenuOpen;
100
101 let tabIndex;
102 if (!props.disabled) {
103     tabIndex = tabIndexProp !== undefined ? tabIndexProp : -1;
104 }
105
106 return (
107     <div
108         {...ContainerProps}
109         ref={containerRef}
110         onFocus={handleFocus}
111         tabIndex={tabIndex}
112         onMouseEnter={handleMouseEnter}
113         onMouseLeave={handleMouseLeave}
114         onKeyDown={handleKeyDown}
115     >
116         <MenuItem
117             {...MenuItemProps}
118             data-open={!!open || undefined}
119             className={className}
120             ref={menuItemRef}
121             keepOpen={keepOpen}
122         >
123             {label}
124             <div style={{ flexGrow: 1 }} />
125             {rightIcon}
126         </MenuItem>
127         <Menu
128             hideBackdrop
129             style={{ pointerEvents: 'none' }}
130             anchorEl={menuItemRef.current}
131             anchorOrigin={{
132                 vertical: 'top',
133                 horizontal: rightAnchored ? 'left' : 'right',
134             }}
135             transformOrigin={{
136                 vertical: 'top',
137                 horizontal: rightAnchored ? 'right' : 'left',
138             }}
```

```
139         css={customTheme}
140         open={!open}
141         autoFocus={false}
142         disableAutoFocus
143         disableEnforceFocus
144         onClose={() => {
145             setIsSubMenuOpen(false);
146         }}
147     >
148     <div ref={menuContainerRef} style={{ pointerEvents: 'auto' }}>
149         {children}
150     </div>
151 </Menu>
```

Step 2: Define the dropdown component

Next, let us define a generic `Dropdown` that we will use later for creating the nested dropdowns. Create a React component called `Dropdown.tsx` with the following contents.

```
1  import * as React from 'react';
2  import styled from '@emotion/styled';
3  import Menu from '@mui/material/Menu';
4  import MenuItem from '@mui/material/MenuItem';
5  import NestedMenuItem from "../NestedMenuItem"
6
7  export const Dropdown = React.forwardRef(
8    (
9      {
10        trigger,
11        menu,
12        keepOpen: keepOpenGlobal,
13        isOpen: controlledIsOpen,
14        onOpen: onControlledOpen,
15        minWidth,
16      },
17      ref
18    ) => {
19      const [isInternalOpen, setInternalOpen] = React.useState(null);
20
21      const isOpen = controlledIsOpen || isInternalOpen;
22
23      let anchorRef = React.useRef(null);
24      if (ref) {
25        anchorRef = ref;
26      }
27
28      const handleOpen = (event) => {
29        event.stopPropagation();
30
31        if (menu.length) {
32          onControlledOpen
33            ? onControlledOpen(event.currentTarget)
34            : setInternalOpen(event.currentTarget);
35        }
36      };
37
38      const handleClose = (event) => {
39        event.stopPropagation();
40
41        if (
42          anchorRef.current &&
43          anchorRef.current.contains(event.target)
44        ) {
45          return;
```

```
46         }
47
48         handleForceClose();
49     };
50
51     const handleForceClose = () => {
52         onControlledOpen
53         ? onControlledOpen(null)
54         : setInternalOpen(null);
55     };
56
57     const renderMenu = (menuItem, index) => {
58         const { keepOpen: keepOpenLocal, ...props } = menuItem.props;
59
60         let extraProps = {};
61         if (props.menu) {
62             extraProps = {
63                 parentMenuOpen: isOpen,
64             };
65         }
66
67         return React.createElement(menuItem.type, {
68             ...props,
69             key: index,
70             ...extraProps,
71             onClick: (event) => {
72                 event.stopPropagation();
73
74                 if (!keepOpenGlobal && !keepOpenLocal) {
75                     handleClose(event);
76                 }
77
78                 if (menuItem.props.onClick) {
79                     menuItem.props.onClick(event);
80                 }
81             },
82             children: props.menu
83             ? React.Children.map(props.menu, renderMenu)
84             : props.children,
85         });
86     };
87
88     return (
89         <>
90             {React.cloneElement(trigger, {
91                 onClick: isOpen ? handleForceClose : handleOpen,
92                 ref: anchorRef
```



```

92         {item.anchorKey,
93       }}}
94
95       <Menu
96         PaperProps={{ sx: { minWidth: minWidth ?? 0 } }}
97         anchorEl={isOpen}
98         open={!isOpen}
99         onClose={handleClose}
100       >
101         {React.Children.map(menu, renderMenu)}
102       </Menu>
103     </>
104   );
105 }
106 );
107
108 export const DropdownMenuItem = styled(MenuItem)`
109   display: flex;
110   justify-content: space-between !important;
111
112   & > svg {
113     margin-left: 32px;
114   }
115 `;
116
117 export const DropdownNestedMenuItem = styled(NestedMenuItem)`
118   display: flex;
119   justify-content: space-between !important;
120

```

Notice that the above code snippet defines `Dropdown`, `DropdownMenuItem`, and `DropdownNestedMenuItem`.

Step 3: Use the Dropdown component to define nested menus

Finally, let us use the `Dropdown` component defined above to define nested dropdown menus. In your `App` component add the following code snippet to define a single `Dropdown` menu with nested items.

Creating Google Docs Menu bar

We can use multiple `Dropdown` menus to create a menu bar similar to Google docs. You can use the following code snippet for it.

Enjoyed the story?

- Subscribe for free to get notified when I publish a new story.
- Want to read more than 3 free stories a month? — Become a Medium member for 5\$/month. You can support me by using my referral link when you sign up. I'll receive a commission at no extra cost to you.

Full Stack

React

Material Ui

Dropdown

UI Design

Reactjs



Follow

Published in Geek Culture

32K Followers · Last published Aug 31, 2023

A new tech publication by Start it up (<https://medium.com/swlh>).

Follow

Written by Utkarsha Bakshi

922 Followers · 100 Following

Demo

Checkout my Markdown Editor at mdedit.ai

The code sandbox embedded below shows the demo app running with nested menus. You can checkout the full source code by clicking the code sandbox link.

No responses yet



What are your thoughts?

Respond

More from Utkarsha Bakshi and Geek Culture



In Geek Culture by Utkarsha Bakshi

How to Upload File to Google Drive from Linux Command Line

Using a command line tool could be useful while uploading large files to Google Drive. In this short post, I will walk you through the...



Dec 26, 2021

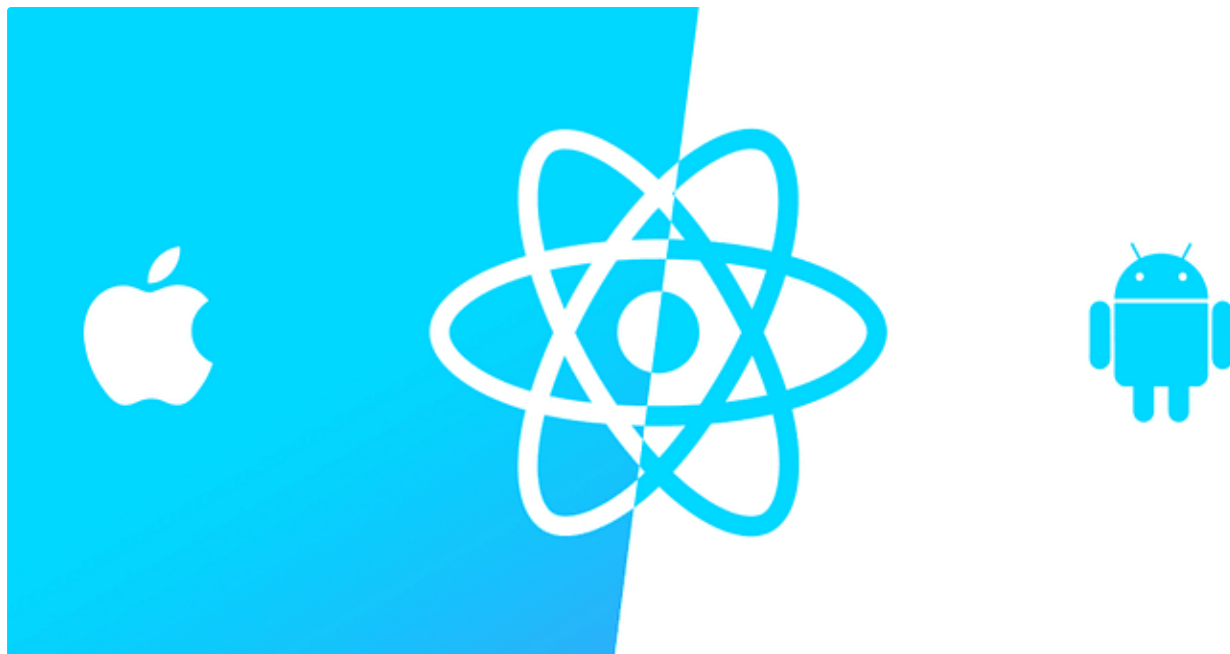


172



9





 In Geek Culture by Anshul Borawake

React Native Generate APK — Debug and Release APK

Generate Debug and Release APK in React Native; Windows, iOS and Linux

Apr 3, 2021  1.8K  15





In Geek Culture by Shu Ishida

Installing Linux (Ubuntu 20.04) on an external portable SSD and pitfalls



In Geek Culture by Utkarsha Bakshi

How to Dockerize a Python AWS Lambda Function

In this post, we will learn how to dockerize a Python AWS Lambda function and deploy it to an AWS account. In a previous article, we...



Jan 4, 2023



79

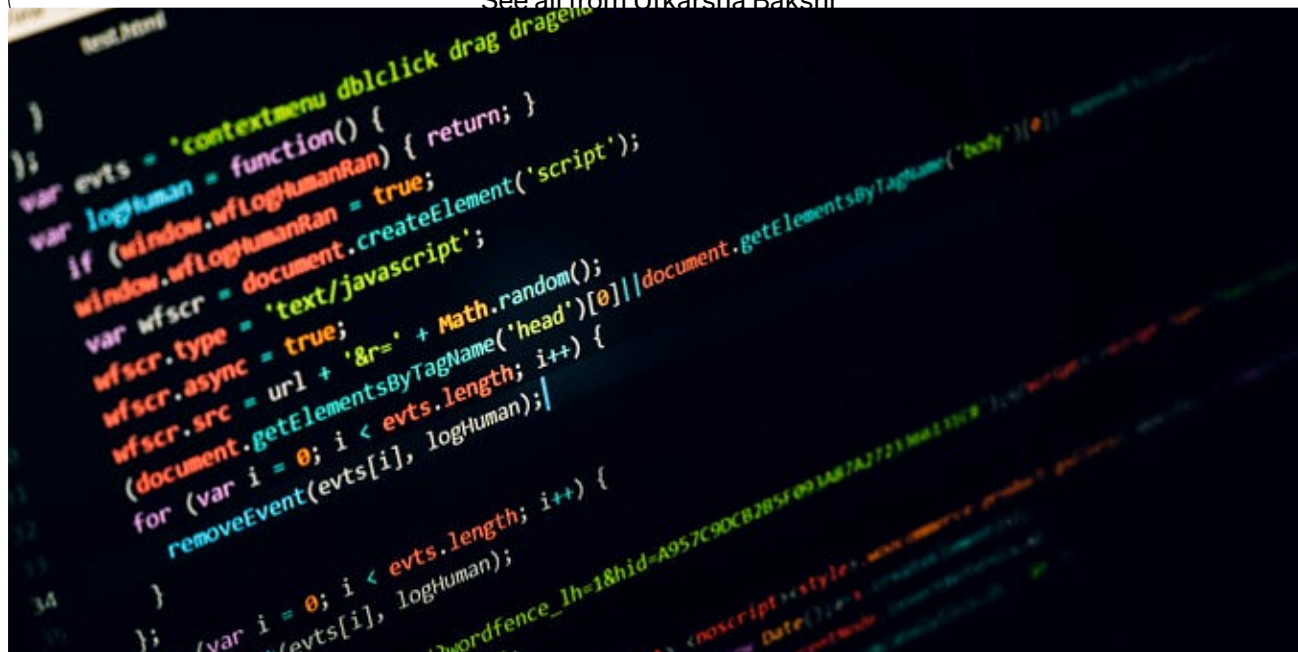


5



Recommended from Medium

See all from Utkarsha Bakshi



JS In JavaScript in Plain English by Digvijay Mahapatra

The Popover API: Building Modals Has Never Been Easier

How this HTML API is replacing hundreds of lines of UI Component code

★ 6d ago 🖱️ 413 💬 5



 In UX Collective by Daniel Berryhill

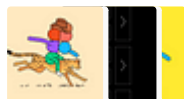
Maybe it's time to let the floating labels die

Floating labels offer no significant value to the user, inject needless confusion, and are ultimately pointless.

★ Feb 25, 2024 🖱️ 551 💬 6



Lists



Figma 101

7 stories · 809 saves



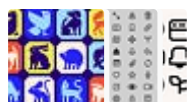
Stories to Help You Grow as a Software Developer

19 stories · 1554 saves



General Coding Knowledge

20 stories · 1861 saves



Icon Design

38 stories · 479 saves

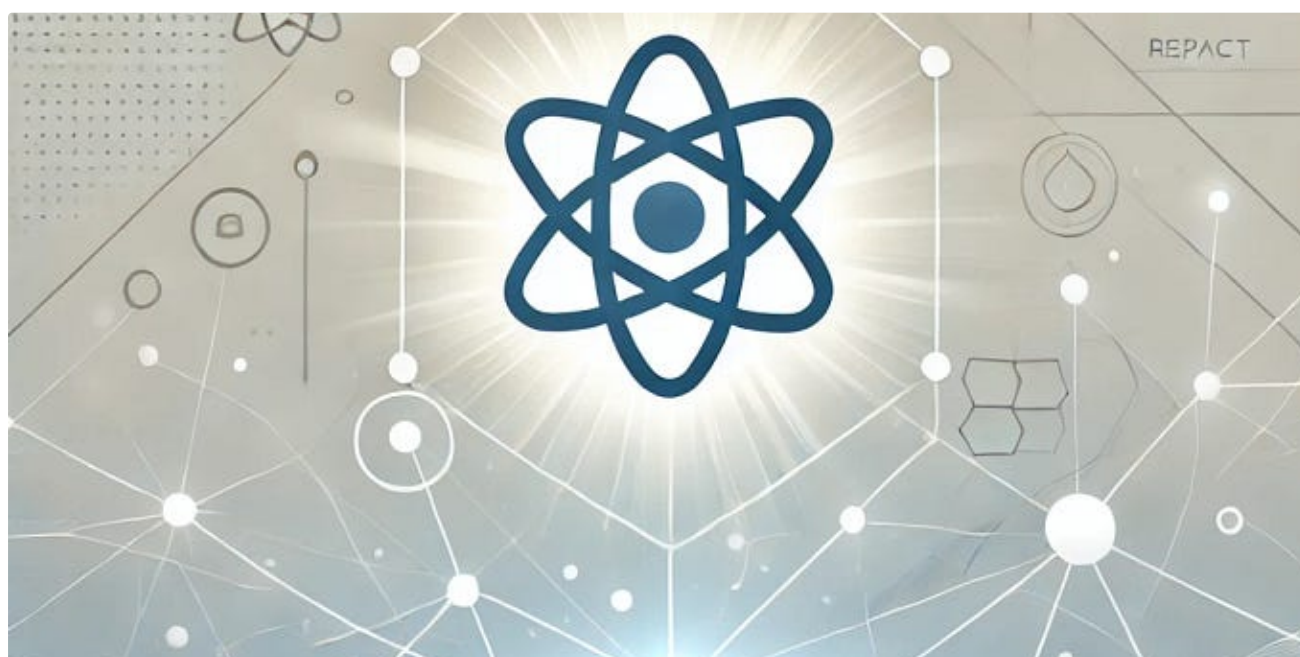


krishna chaitanya

Day 13 — Creating Multi-Step Forms with Validation in React Native

Introduction: On Day 13 of my React Native journey, I tackled multi-step forms with advanced validation. Multi-step forms are a common...

★ Nov 22, 2024





Nayeem Islam

React Router 7: The Ultimate Guide to the New Features and Framework Capabilities

Explore the transformative updates in React Router 7 and learn how to leverage its new features, framework capabilities, and code...

Nov 28, 2024 🖱️ 124 💬 4



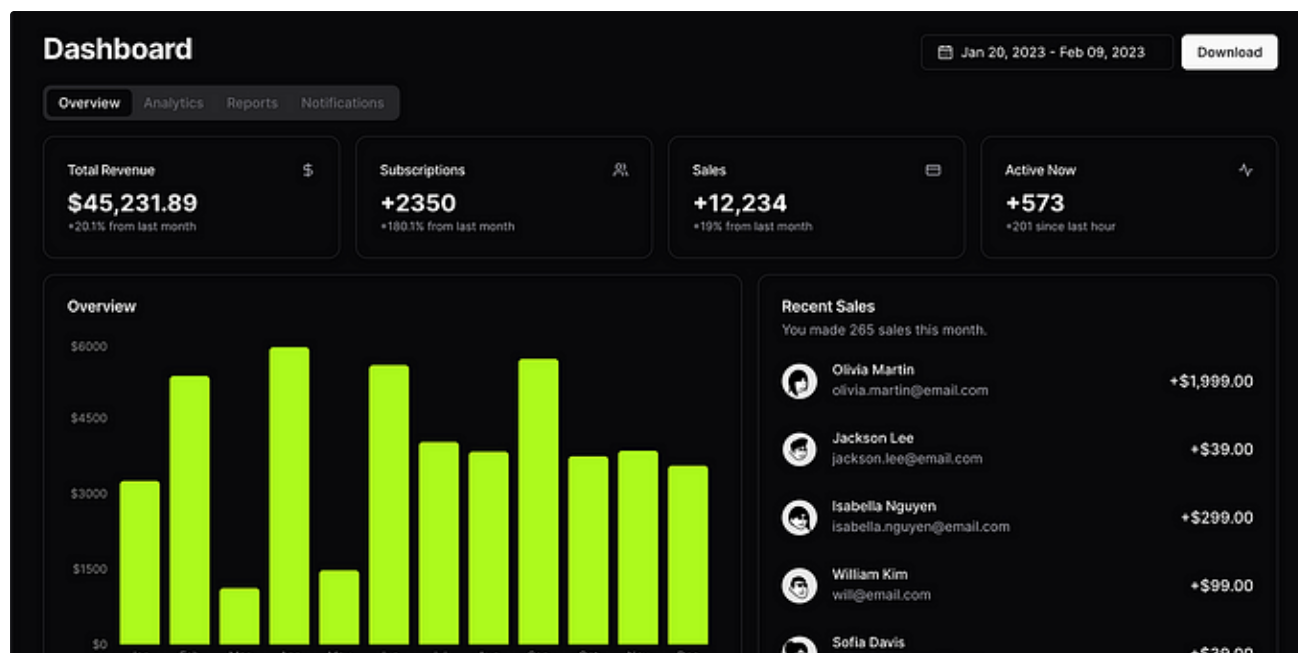
The Expert Developer

Ways to Improve the React Native Performance-(2025)

Developing a high-performing React Native app is essential for providing users with a seamless and responsive experience. Whether you're...

★ Jan 2 🖱️ 51





Shehzad Ahmed

These Radix-based UI Libraries are Crazy Awesome

In the ever-evolving landscape of web development, building accessible and flexible UI components is crucial. Radix UI is an open-source...

Jul 28, 2024 🖱️ 1



See more recommendations